

Atividade 4

Análise léxica com Flex e introdução ao Bison

Questão 1

Um aluno escreveu este arquivo .l:

```
1 %option noyywrap
2 %{
3 #include <stdio.h>
4 %}
5 %%
6 [a-zA-Z_][a-zA-Z0-9_]* { printf("ID(%s)\n", yytext); }
7 "if" { printf("IF\n"); }
8 "ifelse" { printf("IFELSE\n"); }
9 [ \t\n]+ { }
10 %%
```

O flex gera o analisador, mas avisa duas vezes:

```
q1.1:7: warning, rule cannot be matched
q1.1:8: warning, rule cannot be matched
```

Com a entrada

```
ifelse
```

o programa imprime:

- (A) IF, depois ID(else)
- (B) ID(ifelse)
- (C) IFELSE
- (D) IF, ID(else) e IFELSE, porque as três regras casam com a entrada
- (E) nada, porque o flex não gera código para regras que emitiram aviso

Questão 2

```
%option noyywrap
%{
#include <stdio.h>
%}
%%
[0-9]+ { printf("NUM(%s)\n", yytext); }
[ \t]+ { }
. { printf("ERRO(%s)\n", yytext); }
```

```
%%
```

A entrada é uma linha única, terminada por quebra de linha:

```
10 20
```

Qual é a saída?

- (A) NUM(10), NUM(20) e ERRO com a quebra de linha
- (B) NUM(10) e NUM(20), e mais nada: a quebra de linha não casa com regra nenhuma, então o Flex a descarta
- (C) NUM(10), NUM(20) e uma linha em branco, porque a quebra de linha não casa com regra nenhuma e a regra padrão do Flex copia o caractere para a saída
- (D) o flex se recusa a gerar o analisador enquanto faltar uma regra para `\n`
- (E) NUM(10), NUM(20) e um loop infinito na quebra de linha

Questão 3

```
%option noyywrap
%{
#include <stdio.h>
%}
%x COMENTARIO
%%
"/*"                { BEGIN(COMENTARIO); }
<COMENTARIO>"*/"    { BEGIN(INITIAL);    }
[a-z]+              { printf("ID(%s)\n", yytext); }
[ \t\n]+            { }
%%
```

Entrada:

```
abc /* nota */ xyz
```

Saída real do programa:

```
ID(abc)
nota ID(xyz)
```

Qual explicação está certa?

- (A) `COMENTARIO` foi declarado com `%x`, então dentro dele valem apenas as regras marcadas com `<COMENTARIO>`. O texto " `nota` " não casa com nenhuma delas, e a regra padrão do Flex o copia para a saída
- (B) a regra `[a-z]+` continua ativa dentro de `COMENTARIO`, então foi ela que imprimiu "`nota`"
- (C) `BEGIN(COMENTARIO)` só passa a valer no token seguinte, então " `nota` " ainda foi lido no estado `INITIAL`
- (D) o Flex imprime o conteúdo de comentários de propósito, para ajudar na depuração do analisador
- (E) trocar `%x` por `%s` resolve, porque com `%s` o conteúdo do comentário passa a ser descartado

Questão 4

Um arquivo `.l` tem, no seu conjunto de regras, esta:

```
[0-9]+ { yylval = atoi(yytext); return NUM; }
```

Uma desenvolvedora trocou por:

```
[0-9]+ { return atoi(yytext); }
```

O `make` roda sem erro e o executável é gerado. Com a entrada

```
10 20
```

o programa imprime:

```
syntax error
```

Por quê?

- (A) o valor devolvido pelo `return` é o código do token que o léxico entrega ao parser. Devolvendo 10, o léxico diz que leu o token de código 10, que não é `NUM`, e o parser não tem produção que aceite esse token
- (B) o parser reconheceu `NUM` normalmente, mas `$1` ficou com lixo, e o erro vem da ação que imprime a soma
- (C) `atoi` devolve um valor grande demais para o tipo de `yylval`, e o parser detecta o estouro
- (D) faltou declarar `%union` no arquivo `.y`: sem ela nenhum valor passa do léxico para o parser
- (E) a troca não muda nada para o parser, o erro vem de outro ponto do arquivo

Questão 5

O `calc.y` que fizemos na sala de aula (pasta `exemplo2` do repositório no GitHub) declara, nesta ordem:

```
%left  MAIS MENOS
%left  VEZES DIVISAO
%right POT
```

POT é o operador `**`. Para a entrada

```
2 ** 3 ** 2 - 1
```

o programa imprime:

(A) = 511

(B) = 63

(C) = 64

(D) = 512

(E) = 8

Questão 6

Partindo do mesmo `calc.y` da questão anterior, alguém trocou o que as ações fazem: em vez de calcular, elas imprimem.

```
expr : NUM          { printf("%g ", $1); }
    | expr MAIS expr { printf("ADD ");   }
    | expr VEZES expr { printf("MUL ");   }
    | ...
```

As declarações `%left` e `%right` continuam iguais. Para a entrada

```
2 + 3 * 4
```

a saída é:

(A) 2 3 4 MUL ADD

(B) ADD 2 MUL 3 4

(C) 2 ADD 3 MUL 4

(D) 2 3 MUL 4 ADD

(E) nada, porque as ações só rodam depois que o `yyparse` termina de ler a entrada inteira

Questão 7

O Makefile da aula é este:

```
calc: calc.y calc.l
    bison -d calc.y
    flex calc.l
    gcc calc.tab.c lex.yy.c -lm -o calc
```

Tirando o `-d` da primeira linha, o `gcc` para com:

```
calc.l:2:10: fatal error: 'calc.tab.h' file not found
```

O que a opção `-d` faz?

- (A) liga o modo de depuração do parser, que passa a imprimir os estados visitados durante a análise
- (B) gera o `calc.tab.h` com os códigos dos tokens declarados em `%token`. O `calc.l` dá `#include` nesse arquivo, então ele precisa existir antes da compilação
- (C) gera o `calc.output`, com a tabela de estados e a lista de conflitos da gramática
- (D) faz o bison gerar a função `main()`, que sem a opção fica por conta do programador
- (E) coloca os códigos dos tokens como `#define` dentro do próprio `calc.tab.c`, o que dispensa o arquivo `.h`

Questão 8

Veja o lexer a seguir:

```
%option noyywrap
%{
#include <stdio.h>
%}
%%
[0-9]+\.[0-9]+ { printf("REAL(%s)\n", yytext); }
[0-9]+        { printf("INT(%s)\n", yytext); }
"."           { printf("PONTO\n"); }
[ \t\n]+      { }
.             { printf("ERRO(%s)\n", yytext); }
%%
```

Entrada:

```
3.14 3. .5 42
```

- (a) Escreva a saída exata, uma linha por vez.
- (b) O trecho `"3."` não virou um `REAL`. Diga quais regras casaram nessa posição, com que tamanho cada uma casou, e qual critério do Flex decidiu o resultado.

- (c) No ponto solto da entrada, duas regras casam com o mesmo tamanho. Diga quais são e qual delas vence. O critério aqui é o mesmo da letra (b)?
- (d) Altere o `.1` para que `"3."` e `".5"` também sejam reconhecidos como `REAL`, sem estragar nenhum dos outros casos desta entrada. Escreva as regras novas e diga se a posição delas dentro do arquivo muda o resultado. Nas letras (b) e (c) a ordem foi decisiva; aqui ela também é? Justifique.